GUIA TÉCNICO | 09

Avaliação neural, baselines e comparações justas

Capítulo 09/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Separar treino, validação e teste e comparar com modelos de referência.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Não escolher arquitetura, época ou limiar olhando o teste. Acompanhar o gap de generalização e relatar variação entre sementes. Em problemas desbalanceados, a acurácia pode ocultar falhas; avaliar precision, recall e PR-AUC conforme objetivo.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: gap = loss_validação − loss_treino

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Uma avaliação válida distingue treino, validação e teste e usa métricas pertinentes ao desbalanceamento, sem ajustar o modelo ao teste.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
X,y=load_breast_cancer(return_X_y=True)
a,b,ya,yb=train_test_split(X,y,random_state=9,stratify=y,test_size=.25)
model=make_pipeline(StandardScaler(),LogisticRegression(max_iter=500)).fit(a,ya)
print(classification_report(yb,model.predict(b),digits=3))
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de

modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Avaliação em conjunto de teste separado com baseline

Uma avaliação válida distingue treino, validação e teste e usa métricas pertinentes ao desbalanceamento, sem ajustar o modelo ao teste.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import balanced_accuracy_score, confusion_matrix,  ↪
    classification_report
X,y=make_classification(n_samples=240,n_features=8,n_informative=5,weights=[.75,.25],rando  ↪
    m_state=17)
Xt,Xe,yt,ye=train_test_split(X,y,test_size=.25,stratify=y,random_state=17)
base=DummyClassifier(strategy='most_frequent').fit(Xt,yt)
net=make_pipeline(StandardScaler(),MLPClassifier(hidden_layer_sizes=(12,),max_iter=400,ran  ↪
    dom_state=17,early_stopping=True))
net.fit(Xt,yt)
for name,model in [('baseline',base),('MLP',net)]:
    pred=model.predict(Xe)
    print(name,'acurácia balanceada',round(balanced_accuracy_score(ye,pred),3),'matriz',co  ↪
    nfusion_matrix(ye,pred).tolist())
assert confusion_matrix(ye,net.predict(Xe)).shape==(2,2)
```

Acurácia simples pode esconder resultados ruins em classe minoritária; interpretar matriz, recall e incerteza em amostra reduzida.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que não ajustar o limiar de decisão no teste?

Resposta: Porque escolher o limiar no teste contamina a estimativa de desempenho fora da amostra.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide